

Drones for Fast, Low-Cost 3D Large Photorealistic Indoor Scene Reconstruction

Shiran Yuan Stephen Lee May Liu Brayton Lordianto
Junming Chen
UC Berkeley

Advised by Avidesh Zakhor

Word Count: 5044

Editor: Stephen Lee

Project ID: 32

Contents

1	Introduction - SL	2
2	Literature Review	3
2.1	Previous work on drones and 3DGS - JC	3
2.2	Mesh extraction - SY	3
3	Approach and Experiments	4
3.1	Existing Pipeline	4
3.1.1	Data Acquisition and Image Processing - ML	4
3.1.2	Divide-and-Conquer Strategy for Large-Scale Scenes - ML	4
3.1.3	Recovering Camera Poses and Creating a Point Cloud - BL, ML	5
3.1.4	Scene Reconstruction - ML	5
3.2	Pipeline Improvements	6
3.2.1	COLMAP-free Reconstruction Methods - ML, BL	6
3.2.2	Mesh Extraction - ML, BL	8
3.2.3	Mesh Post-Processing - BL, SL	9
3.2.4	Novel-view Synthesis - SL, SY, JC	11
4	Discussion	14
4.1	Extension - JC, SL	14
4.2	Limitations - SY	15
5	Conclusion - SY, BL (2nd/3rd pg)	15
6	Acknowledgments - SL	15

Drones for Fast, Low-Cost 3D Large Photorealistic Indoor Scene Reconstruction

Shiran Yuan Stephen Lee May Liu Brayton Lordianto Junming Chen
UC Berkeley

{shiran_yuan, stephen_lee129, mayliujj, brayton.lordianto, leochenjm}@berkeley.edu

Abstract

In our increasingly digital world, creating digital environments for exploration and simulation has become an exciting means of sharing our physical environments with each other digitally. Our team has refined a pipeline for quickly recreating physical spaces using drones at low cost. This pipeline leveraged 360° cameras and drones to easily collect data, and 3D Gaussian Splatting (3DGS) to efficiently recreate photorealistic indoor scenes, rendered in real-time. We extend the capabilities of these reconstructed scenes by extracting meshes from our 3DGS representations. This allows our model to be applicable to a far wider range of uses such as virtual assets for robotics simulation, AR/VR, and game development.

1 Introduction

Reconstructing scenes from a stream of captured images has been a popular topic in computer vision for quite some time. Recently popular methods such as Neural Radiance Fields (NeRFs) [7] have demonstrated highly photorealistic renders of 3D scenes. NeRFs can capture intricate details of lighting, shadows, reflections, and textures, making them excellent for applications such as virtual reality, gaming, and visual effects. However, they come with a high computational cost that makes them expensive and slow to render scenes. 3D Gaussian Splatting (3DGS) [4] has been presented as a method to rectify these drawbacks that NeRFs face.

Rather than encoding the scene as a neural network, 3DGS directly represents the scene as a collection of Gaussian Splats, which can be thought of as colored blobs that represent points within the scene. This allows 3DGS to produce much faster, real-time renders of the reconstructed scene, while still maintaining high quality photorealism.

There are a few main limitations 3DGS faces that we will discuss:

1. Acquiring data can be difficult and expensive
2. 3DGS scene representations are not very accurate for anything other than novel view synthesis

The problem of data acquisition has been addressed in prior work by using drones to traverse spaces and a 360 degree camera mounted on top to collect a continuous stream of images of that traversal [11]. This allows for data capture to be completed within a matter of a few minutes of flight time, rather than having to manually capture images at various locations in the space to reconstruct.

As for widening the use cases for the reconstructed scenes, we present two main improvements: 1) mesh extraction 2) random splitting preprocessing. We also collect new dataset from a variety of scenes. Commonly used software such as MuJoCo for robotics simulation and game engines such as Unity for AR/VR and game development do not support assets that are represented by 3DGS. By applying techniques such as SuGaR [12] and 2DGS [13], we can extract meshes that represent the geometry of our scenes that can be used in these software applications.

Generation of 3D assets for AR/VR, gaming, movies



Digital twin modeling of factories and warehouses



Figure 1: Visualization for some of applications that this technology can be used for; courtesy of [11]

2 Literature Review

2.1 Previous work on drones and 3DGS

For large-scale indoor 3D scene reconstruction, it is time-consuming and difficult to capture a dense set of images manually. The use of drones has thus become more popular in recent years due to their ability to capture images from diverse perspectives and navigate spaces that is inaccessible to ground-based methods. While traditional methods such as Structure-from-Motion (SfM) [8] and Multi-View Stereo (MVS) can reconstruct the coarse 3D scene from captured images, they often struggle in large-scale indoor scenes.

To address these challenges, researchers have explored neural implicit surface representations and learning-based reconstruction techniques [9]. Several works have demonstrated the effectiveness of drone-captured images for large-scale indoor scene reconstruction. Li et al. [10] proposed a fully automated pipeline that combines traditional SfM with NeRF. Their approach integrates semantic segmentation for category-wise plane fitting, improving reconstruction quality in these low-texture regions. Recently, 3D Gaussian Splatting [4] has become widely used for scene reconstruction and novel view synthesis. Chen et al. [11] proposed a drone-based pipeline leveraging 3D Gaussian Splatting (3DGS) to efficiently reconstruct large-scale indoor scenes from drone-captured 360° videos. The 3DGS directly models the scene as a set of anisotropic 3D Gaussians, allowing for high-quality novel-view rendering with significantly reduced computation time. They also use a divide-and-conquer strategy to enable parallel processing of individual scene blocks, which further improves the efficiency.

2.2 Mesh extraction

Despite the effectiveness of 3DGS, a key problem that it fails to solve is mesh extraction. Meshes are collections of planar polygons that represent a 3D surface, which are commonly used in fields such as graphics and robotics. They are essential for many tasks such as collision detection (important for simulations) and inverse rendering (for making the scene display photorealistic).

In this project we explore two variants on 3DGS which benefit mesh extraction. One is SuGaR [12], which simultaneously optimizes a triangular, mesh-like structure representing the surface and Gaussians bound to the triangles, thus achieving more accurate surface geometry. The other method is 2DGS [13], which uses 2D planar Gaussians instead of the usual 3D Gaussians, and applies normal estimation as regularization to ensure that the Gaussians align with the surface. Both methods reduce the amount of artifacts and floaters (low-opacity “ghostly” hallucinations that occur in the

reconstructed scene representation due to unavoidable errors in input data), making mesh extraction easier and more precise.

We also consider methods for post-processing after the scene representation is turned into a mesh, such as histogramming to remove bump and hole artifacts in the floor.

3 Approach and Experiments

Our approach improves upon previous methods by introducing random splitting in the pre-processing pipeline and incorporating mesh extraction techniques to bridge the gap between image-based reconstruction and explicit geometry representation. We also introduce post-processing techniques to improve the quality of extracted meshes.

3.1 Existing Pipeline

Our study builds upon a scalable 3D reconstruction pipeline [11] that uses drone-captured 360° images combined with 3DGS for indoor novel-view synthesis. In this next section, we elaborate on the details of this pipeline, and some of the more minor modifications that we have made.

3.1.1 Data Acquisition and Image Processing

Similar to Chen et al. [11], we mount a 360° camera (insta360 OneRS Twin Edition) on a drone (DJI Mavic Air 2) to capture video sequences while flying in a simple straight trajectory, which makes piloting easier compared to traditional zig-zag paths. This method allows us to capture diverse viewpoints without adding unnecessary motion blur. The video frames are unwarped into cube-mapped images, which are then used for feature matching in the reconstruction pipeline.

Since the drone itself appears in the images, we apply image inpainting before processing by using the Segment Anything Model (SAM) to mask out the drone body [5], then inpaint the missing regions to keep multi-view consistency.



Figure 2: Our drone with the 360° camera securely mounted on it

3.1.2 Divide-and-Conquer Strategy for Large-Scale Scenes

Last year’s team applied a divide-and-conquer (DAC) approach [11] to split the scene into smaller blocks for scalable processing in large-scale, multi-story indoor environments. Each block was created with 25% overlap with its neighbors, which aided in merging them back into a coherent 3D scene and reduced erroneous feature matching, being particularly useful in settings with repetitive elements such as hallways and ceilings.

Although we did not use DAC in our pipeline this year, we analyzed the dataset generated from that process in 2DGS[13], SuGaR[12], and Mast3r[17] reconstructions and observed some of the same challenges reported in [11]. In particular, we noted that the inclusion of floor and ceiling images tended to degrade reconstruction quality. These regions are often repetitive and lack distinctive features, causing COLMAP to produce inaccurate camera poses. Our experiments confirmed that feeding these images into 3DGS and 2DGS introduced floating artifacts, incorrect alignments, and noisy depth estimations. Consequently, we chose to exclude floor and ceiling images from both the SfM and Gaussian reconstruction stages of our pipeline.

However, we recognize that excluding these views is not a sustainable long-term solution. In future work, we plan to incorporate them back into the pipeline after reconstructing the primary scene structure. By delaying their integration, we aim to reduce the risk of propagating feature-matching errors throughout the entire reconstruction and instead use these images to refine the scene geometry in a controlled, localized manner.

3.1.3 Recovering Camera Poses and Creating a Point Cloud

After applying inpainting and optional blocking to the images, we proceed to generate a point cloud representing the 3D structure of the scene (refer to Structure-from-Motion in Figure 4).

To perform this step, we used COLMAP [16], a state-of-the-art Structure-from-Motion (SfM) system that jointly estimates camera poses and reconstructs sparse 3D points. We applied COLMAP to a new dataset we collected this year, which captures a different indoor environment within Cory Hall. The new dataset contains more visual complexity, including desks, chairs, and equipment, as opposed to the relatively clean, feature-sparse hallway used in last year’s project [11]. This richer scene allows us to better evaluate reconstruction performance in cluttered real-world spaces.

Despite COLMAP’s robust feature matching and bundle adjustment capabilities, we encountered two limitations. First, COLMAP is computationally intensive. In our experiments, it took approximately 2 hours to process 300 images, which is a bottleneck for scaling to larger datasets. Second, COLMAP occasionally produces extraneous 3D points, which result in floater artifacts in the final rendering (see Figure 3). These issues motivate our exploration of alternative reconstruction methods that do not rely on COLMAP, discussed in later sections.



Figure 3: A example of floater artifacts in the digital reconstruction. Notice the floater artifacts at the top edge of the entrance.

3.1.4 Scene Reconstruction

Finally, we describe the 3D Gaussian Splatting (3DGS) pipeline developed by last year’s team [11], which serves as our baseline for comparison. This method reconstructs indoor scenes using anisotropic 3D Gaussians, providing a fast and efficient alternative to volumetric neural representations such as NeRF. The reconstruction begins with Structure-from-Motion (SfM) to estimate camera poses and a sparse point cloud, which are then used to initialize the Gaussian Splatting process.

In contrast, our reconstruction experiments this year use alternative methods, including 2DGS[13], SuGaR[12], MegaSaM[18], and MaSt3r[17], on a different and more visually complex Cory Hall dataset. Throughout the report, we compare the performance and quality of our results against the 3DGS baseline.

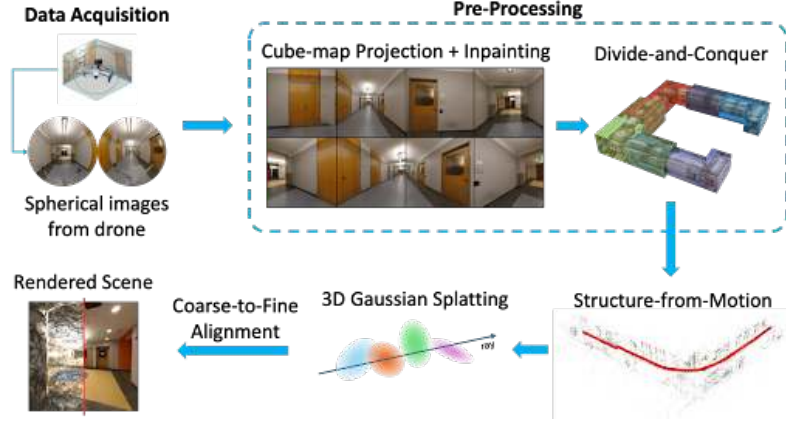


Figure 4: Block diagram outlining the pipeline for taking raw video footage and reconstructing the scene using 3DGS; courtesy of [11]

3.2 Pipeline Improvements

In this next section, we detail the improvements made to the pipeline in the previous section. These include COLMAP-free reconstruction methods and random splitting to reduce the number of floater artifacts, and mesh extraction methods to expand the applicability of the scenes we reconstruct. Sections 3.2.1 and 3.2.3 include work done by Brayton and May for their CS280 project.

3.2.1 COLMAP-free Reconstruction Methods

To address the limitations of COLMAP, particularly its reliance on sparse feature matching, long runtimes (up to 2 hours for 300 images), and sensitivity to textureless or repetitive indoor scenes, we explored alternatives that do not depend on traditional SfM. We focused on machine learning based methods that are more robust in low-texture, cluttered environments and offer faster processing.

Specifically, we experimented with Mast3R [17] and MegaSaM [18], two recent learning based approaches for estimating camera poses and generating point clouds. Mast3R uses transformer based matching and neural surface optimization to find dense correspondences between views, even in scenes with weak visual features. Unlike traditional pipelines that depend on keypoints, Mast3R learns how each point in one image should map to another using geometry aware features. However, in practice, we found that Mast3R is computationally expensive. It took around 30 minutes to process the same number of images that MegaSaM completed in under 5 minutes (around 100 front facing images in a single Cory Hall block). Its outputs were also inconsistent and noisy, especially in areas without strong texture or lighting.

In contrast, MegaSaM performed much better on our Cory Hall dataset, both in terms of speed and reconstruction quality. MegaSaM runs a SLAM style pipeline that combines learned monocular depth estimation with factor graph optimization to recover dense depth maps and camera trajectories. Instead of relying on feature matching across wide baselines, it tracks geometry from frame to frame. This makes it well suited for indoor environments with repeated structures, limited movement paths, and weak texture.

The difference of the point clouds reconstructed can be found in comparing Figures 5, 6, and 7. Mast3R not only takes longer but also reconstructs point clouds that have large gaps and lack of detail. We observed that Mast3R’s rendered outputs exhibit significant artifacts and geometric inconsistencies, particularly in textureless or cluttered indoor scenes such as Cory Hall (Figure 5). This degradation can be attributed to MAST3R’s reliance on transformer-based feature matching and neural surface optimization, which often fail to converge cleanly in the absence of strong geometric priors. In contrast, MegaSaM leverages structured camera motion and dense correspondence estimation, enabling it to generate more stable and complete reconstructions under the same data conditions.



(a) Side view

(b) Top view

Figure 5: A reconstruction of one block using MaSt3R.

To run MegaSaM on the entire Cory Hall third floor dataset, we compiled a unified set of 400 front-facing images aggregated from four separate drone-captured blocks of Cory Hall. These images were selected to align with the camera motion direction, ensuring sequential consistency required by MegaSaM’s SLAM-based pose optimization. To accommodate the scale of our dataset, we modified the original MegaSaM Python code to enable batch-wise processing of image and depth sequences, which allowed us to handle longer trajectories and larger memory footprints without runtime failures. Importantly, no parameter tuning was required; the entire pipeline was executed using MegaSaM’s default configuration. The final output was a structured .npz file containing RGB frames, intrinsics, depth maps, and trajectories, which would serve as the foundational reconstruction used for downstream evaluation and visualization.



Figure 6: Side view of MegaSaM reconstruction

After running MegaSaM, we used a custom script to convert the .npz outputs into .ply point clouds. This involved extracting RGB, depth maps, intrinsics, and poses, and reprojecting each valid depth pixel into 3D space using camera parameters. The resulting point clouds were visualized using Open3D. We observed clear spatial layout, sharp edges, and good color alignment (Figure 6). We attribute this fidelity to MegaSaM’s view-dense, scene-sparse nature. Unlike traditional SfM pipelines that rely on sparse feature correspondences across wide baselines, MegaSaM processes dense monocular depth predictions over a densely sampled input video stream. This tight temporal



Figure 7: Front view of MegaSaM full Cory Hall third floor reconstruction

continuity enables it to build consistent geometry frame-by-frame with strong local context, even in regions where global feature matching would fail. Structural details like walls, doors, and floor were all well preserved. All of this was achieved without any use of COLMAP’s triangulation or feature based matching.

That said, MegaSaM also has its limitations. It assumes smooth, forward facing camera motion from sequential frames. This means it does not capture full 360 degree geometry or handle backward facing surfaces well. Occluded and unobserved regions are often missing or incomplete.

While MegaSaM successfully captured the overall layout of the third floor of Cory Hall, the resulting point cloud did not reflect the true, rectilinear U-shaped structure of the building. Instead, the reconstructed hallway exhibits a noticeable wobble, curving inward or outward in places where the real-world architecture is straight (Figure 7). We hypothesize this deviation is caused by cumulative drift in pose estimation and the lack of global geometric constraints to enforce linearity over long spatial extents. Since MegaSaM integrates depth locally from a monocular prior and optimizes poses incrementally, it lacks mechanisms to anchor the trajectory against long-range structural regularities (e.g., straight walls or parallel corridors). This is in contrast to methods like COLMAP, which can exploit wide-baseline correspondences to correct global drift and enforce rigid geometry over large distances. That said, for our dataset and trajectory, this tradeoff was acceptable.

Overall, we found that MegaSaM outperformed both Mast3R and COLMAP in terms of speed, reconstruction consistency, and robustness to real world indoor scenes. Our results suggest that COLMAP free pipelines based on learned depth and sequential tracking are a promising direction.

3.2.2 Mesh Extraction

To obtain explicit geometry from our reconstruction, we incorporate 2D Gaussian Splatting (2DGS, Huang et al., 2024) and SuGaR (Supervised Gaussian Regularization, Guédon and Lepetit, 2024) to extract meshes from the reconstructed point-based representation. While these methods help us get an initial mesh, the extracted geometry is suboptimal – it lacks clean surfaces, has noisy edges, and contains many angular floating artifacts as seen in Figure 8. This is why we focus on post-processing the mesh to refine its quality in the next section.



Figure 8: Example of an unprocessed mesh extracted using 2DGS. Notice the large dips in the floor, holes in surfaces, and angular floating artifacts circled in red.

3.2.3 Mesh Post-Processing

As demonstrated, there are many problems with the meshes extracted from 2DGS and SuGaR. In this section, we discuss our implementations of post-processing techniques to minimize the inaccuracies and inconsistencies of the mesh ¹. The extracted mesh has artifacts, incomplete surfaces, and inconsistent geometry, so we focus on smoothing, hole-filling, and mesh simplification. We also test iterative refinement methods to improve structure consistency across blocks. Note that some of these artifacts are caused by floaters, and we can easily fix them in the mesh. This fix means that the floaters do not affect the quality of the resulting mesh. The goal is to make the mesh more usable for downstream applications such as VR rendering, simulation, and digital twin modeling. Unlike typical NeRF-based methods that struggle with explicit geometry extraction, our approach combines Gaussian Splatting with mesh refinement to make the output more practical.

Our current strategy incorporates the detection of floors and ceilings via histogramming, and then apply a projection of extraneous mesh elements onto the said floors and ceilings. This first step results in a histogram such as demonstrated in Figure 9, in which we can detect the floors and ceiling y values, as they are the peaks of histogrammed values. This is true because the floor and ceiling will have most points relative to anywhere else in an indoor space. In this example, for instance, we roughly have the floor to be $y = -1.5$ and the ceiling to be at $y = -0.4$. The visual results of this histogramming can be observed in figure 10, where we can clearly see the large dips in the floor have been removed, giving much more accurate scene geometry.

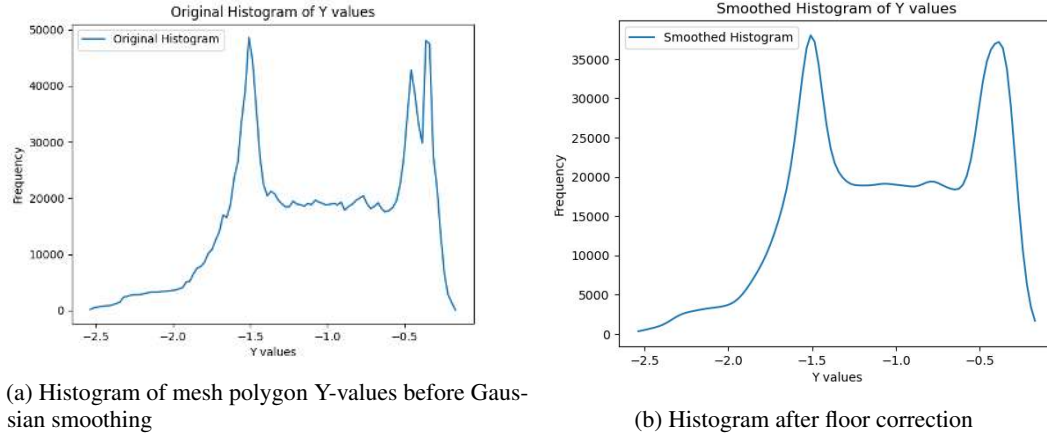


Figure 9: Statistical verification of floor plane coordinates through histogram analysis. The before histogram shows a raw histogram of y values, while the right shows a smoothed histogram of these values.

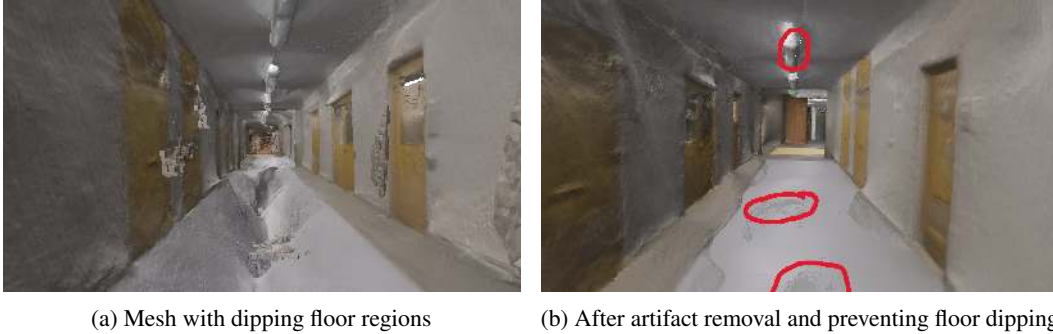


Figure 10: Floor correction process showing the identification and resolution of dipping floor regions. (a) shows problematic areas where floors incorrectly dip below expected elevation levels, while the (b) demonstrates the corrected mesh with properly aligned floor surfaces after processing.

¹<https://github.com/Brayton-Lordianto/3D-Indoor-Digital-Reconstruction-Utilities.git>

Another major issue with the meshes we extracted had to do with the floater artifacts that resulted from the Gaussian Splatting. These floaters would also be extracted as mesh components, which is undesirable. To remove these undesirable irregularities, we observe that most of these components are disconnected from the rest of the indoor space mesh, whereas most of the correct mesh reconstruction are connected with each other. The reason that this may be true is that floaters primarily emerge from either areas observed from limited viewpoints which create low-confidence Gaussians that "float" in free space or Gaussians slightly off-surface to approximate specular highlights. These processes inherently create small, isolated components since they don't represent physical structure and thus don't maintain connectivity with the primary architectural elements.

From this, we hypothesize that simply taking the largest connected component of the mesh would yield near artifact-free results. We accomplish this process by first extracting the vertices, indexed faces, and RGB values of the mesh. We can then iterate over each face to determine the mesh connectivity, finding connected components with a simple count of vertices per component. From here we can simply take the largest one. We can see that this strategy is very effective in Figure 11.

This filtering approach implicitly applies a form of global scene reasoning - a continuous enclosed space should form a single manifold surface with boundaries only at scan limits or occluded regions. Any component that fails this fundamental property is likely an artifact rather than a legitimate architectural element. This connectivity-based filtering aligns with how humans intuitively understand indoor spaces as continuous environments rather than collections of disconnected elements, making it both computationally efficient and perceptually sound for architectural reconstruction.

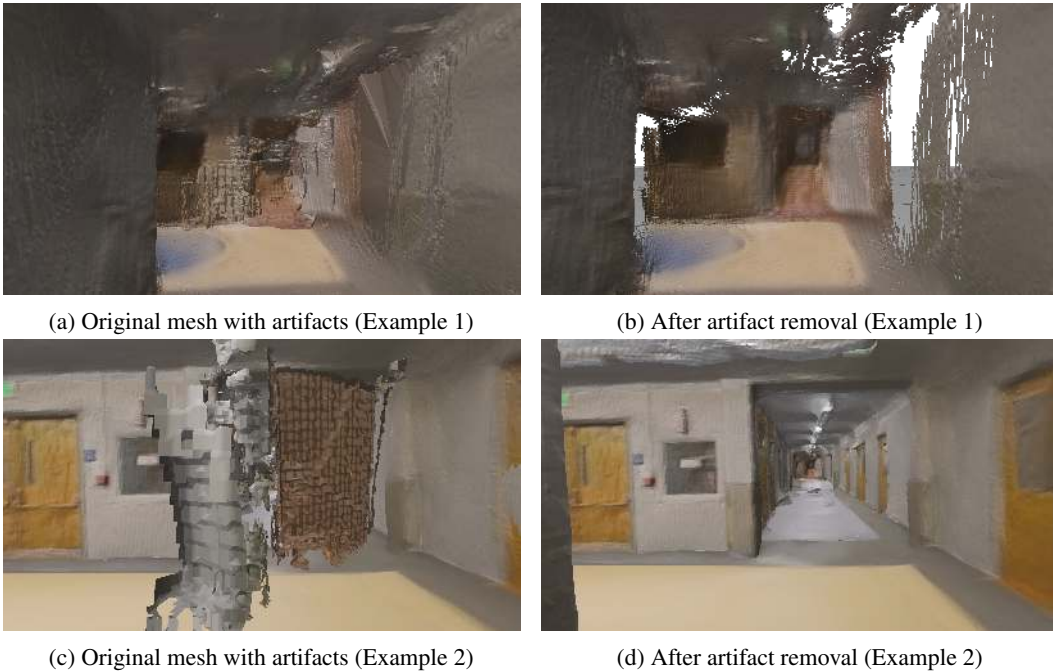


Figure 11: Artifact removal process showing before and after states for two example regions. The filtering algorithm successfully identifies and removes irregularities while preserving the underlying structure.

Finally, we observe that a large number of holes, especially on the walls and floors of the scene, pose a significant challenge for extracting a high-quality mesh. Holes are defined as a loop, or a connected closed path, that consists exclusively of boundary edges. We address this issue by implementing an algorithm proposed by Brunton et al. [19] that adds additional faces with existing vertices. We begin by detecting boundary edges in every hole by looping over all the faces and identify which edge is only called once. We observe that, the same hole must contain another edge that contains the vertex in this edge. Therefore, beginning with edge $E = (u, v)$, we can find the next edge in the hole by looping over remaining boundary edges that contain vertex v . Iterating until we reach the same initial vertex u allows us to retrieve all vertices that comprise the hole.

Now that we have identified the vertices belonging to each hole, we fill it by using Delaunay triangulation to create well-shaped triangles that respect local curvature. To use this method, we must project it in 2D, as it does not work well in 3D. However, this is convenient for our use case, because the holes we are trying to fill are along planar floors and walls, meaning that most of these holes are close to a common plane. This method allows us to create flat patches that respect the planar nature of the actual scene geometry.

We find the best fit plane by computing the singular values of the vertices of the hole to its centroid, each singular value being the direction of greatest variance. If we consider floor holes for example, the directions of largest variance are along the X and Z axes, while Y values would be much more tightly clustered about the ground truth floor Y value. Applying similar logic to the walls, we can project each vertex to its best fit plane, where we can simply run the Delaunay algorithm.

Our implementation completes this operation for 68,195 holes in just 71 seconds, scaling linearly with number of holes. Figure 12 shows the results of this post-processing algorithm.



(a) Mesh with visible holes after artifact removal



(b) After hole filling process (Example 1)



(c) Another region with holes (Example 2). Notice the small cracks on the ceiling lights and the patches on the floor.



(d) After hole filling process (Example 2). Notice the disappearance of the cracks on the ceiling and the filling in of thin primitives on the small patches of the floor.

Figure 12: Hole filling process demonstrating the detection and repair of mesh discontinuities. The algorithm identifies boundary edges and generates appropriate fill geometry while maintaining mesh consistency.

3.2.4 Novel-view Synthesis

Another issue that we are addressing in our approach are floaters and other artifacts that reduce the visual fidelity of our 3D reconstructions. We have introduced a new pre-processing pipeline to introduce more diverse viewing angles to the images that we pass into our pipeline. The pipeline developed by last year's team [11] creates 2 consistent rotations of cubemap projections of each of the spherical image frames, one aligned with the drone, and another rotated 45° from the front of the drone. This method is shown in Figure 13, and will be referred to as "Last Year's Method". Last Year's Method works, but limits the full diversity of viewing angles that are being captured the 360° camera for each frame. We look to improve upon this by introducing random rotations of our cubemap projections to more fully capture the diverse viewing angles that are provided by our camera setup, our initial attempt at this will be referred to as "Naive Random Rotations".

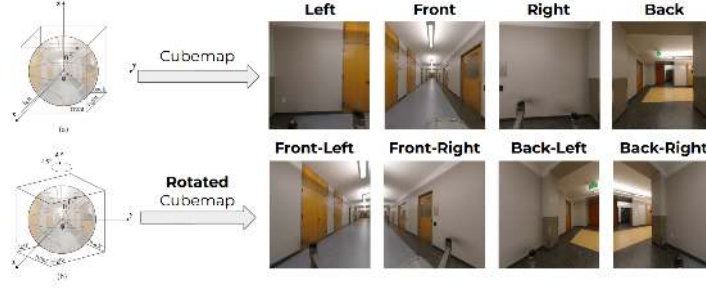
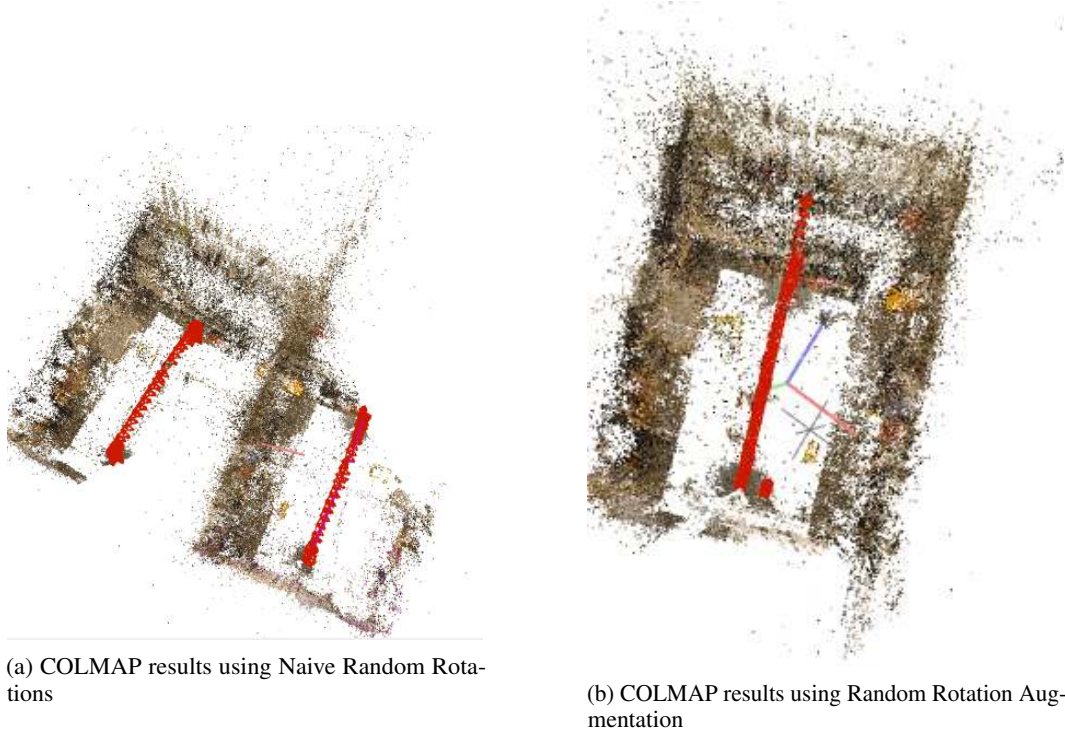


Figure 13: Example views produced by Last Year's Method, courtesy of [11]

Naive Random Rotations makes it much more difficult for COLMAP to resolve correct camera poses along its flight path. We believe that this is primarily due to the impact of distortion. We assume that the panorama are perfect equiangular models, and try to apply random rotation to augment dataset with diverse viewing angles. However, the real captured panorama have distortion and misalignment from the 360° camera, which in reality is actually just stitching two 180° wide-angle images together. We found that this distortion ruins the COLMAP results especially under large rotation angles. We demonstrate the COLMAP results from this Naive Random Rotations method in Figure 14a, where we can see that the scene geometry is distorted and malformed. This motivated an iteration on this method, which will be referred to as "Random Rotation Augmentation". Random Rotation Augmentation resolves the issues faced by Naive Random Rotations by reducing the azimuth and elevation angles to be independently at random from the range $[-10^\circ, 10^\circ]$, rather than purely random. Random Rotation Augmentation produces different, unique viewing angles at each frame along the flight path, which we then used to augment the original pipeline dataset. In total, we take 500 images from the original pre-processing pipeline method, and 500 images with our random rotations to create a 1,000 image dataset for the drone room scene. The COLMAP results of this method can be see in Figure 14b, where we can see the expected layout of the room with the correct flight path taken.



(a) COLMAP results using Naive Random Rotations

(b) COLMAP results using Random Rotation Augmentation

Figure 14: Comparison of COLMAP results from our initial Naive Random Rotations versus the improved Random Rotation Augmentation.

We conduct experiments using 3DGS, 2DGS, and SuGaR on a scene (Drone Room) that we captured to determine the effectiveness of Random Rotation Augmentation. Overall, our results for this are mixed. We consistently found that the Last Year’s Method struggled to capture complex scene details in such a cluttered scene, particularly in the corners of the room that were far from the flight path of the drone. In these areas, we see particularly high numbers of floater artifacts and generally scene geometry that looks somewhat distorted. In contrast, the introduction of additional random views appears in our Random Rotation Augmentation method seem to have successfully reduced the impact of floaters in these parts of the scene. This impact can be seen in Figure 15.



(a) 3DGS using Last Year’s Method. Notice the cameras and lights towards the top of the image have many floaters making them difficult to discern, and the storage rack at the bottom of the image is poorly formed.



(b) 3DGS using Random Rotation Augmentation. Notice that the cameras and lights towards the top of the image are much clearer now, and the storage rack at the bottom of the image has a much clearer shape.



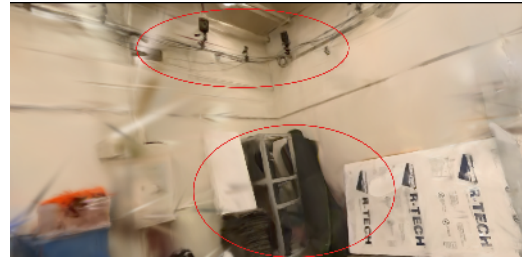
(c) SuGaR using Last Year’s Method. Notice that the large number of blurry floaters in the center of the image that make the details impossible to make out.



(d) SuGaR using Random Rotation Augmentation. Notice that both the storage rack and rolled up carpet in the center of the image can clearly be made out now.



(e) 2DGS using Last Year’s Method. Notice the cameras towards the top of the image have floaters, and the text on the foam board at the bottom of the image is blurred out and warped particularly within the red circle.



(f) 2DGS using Random Rotation Augmentation. Notice that both the cameras at the top of the image and the text on the foam board at the bottom of the image are clearer. We do see more floaters towards the edges of the image however.

Figure 15: We can see that for original splitting in (a), (c), and (e), each reconstruction method has difficulty when trying to render novel views at the corners of the room. In (a), (c), and (e) there are a significant amount of floater artifacts that obscure the details of the scene. Looking at the random rotations, we can see in (b) and (d) there are significantly fewer floaters obstructing the scene details, which are much clearer and more detailed as a whole. In (f), we can see that some scene details such as the cameras and foam board have fewer floaters and are much clearer, but the edges of the scene have more floaters

However, we also found that this method of augmenting the original splitting method with additional randomly rotated cubemap projections introduced new floater artifacts elsewhere, particularly along the flight path of the drone. We demonstrate this in Figure 16, where we can see that the augmented splitting method has many floaters along the flight path of the drone down the center of the room using 3DGS. We observed very similar results for SuGaR and 2DGS as well.

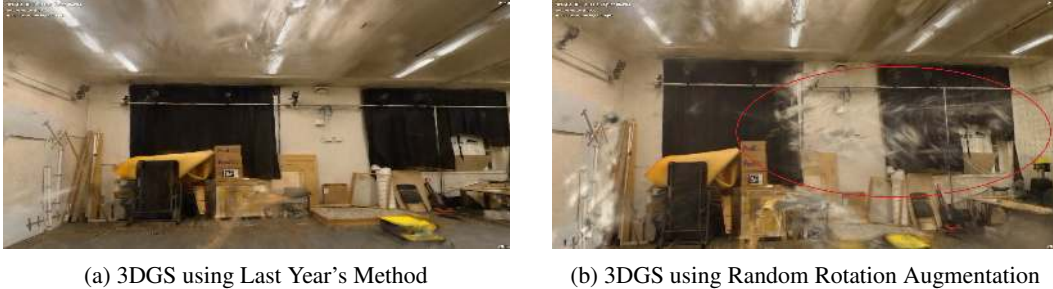


Figure 16: Comparison of splitting methods for 3DGS. We can see that the augmented splitting method introduces new floater artifacts along the flight path of the drone circled in red.

Overall, this random splitting demonstrated mixed results as it reduced the number of floaters and improved reconstruction quality for some views, while it introduced new floaters for other views. The reconstruction methods used (3DGS, SuGaR, and 2DGS) each produced similar quality results, having similar floater issues for each splitting method at similar views. We suspect that the new floaters originate from the random rotations introducing more camera motion between frames making it more difficult for COLMAP to correctly do feature matching. Perhaps gradual, more scripted rotations would be helpful here rather than purely random, as we can better choreograph the rotation of the camera while still introducing more diverse viewing angles. This method would likely necessitate a much larger number of images however, which may prove impractical. In the future, MegaSAM might prove to be a more effective method for this, because as discussed in previous sections, we found it worked well for indoor scenes and is much faster than COLMAP, which may accommodate for the larger input size. Another area for future work is to account for the true nature of the camera. We discussed previously that there is distortion and misalignment present since the 360° camera just stitches two 180° wide-angle images together, which made larger random rotation angles difficult. We believe that the most likely solution to this issue is to setup two separate camera models for both the front and back 180° camera and then stitch them back together.

4 Discussion

4.1 Extension

Our approach demonstrates promising capabilities in 3D large indoor scene reconstruction, and it can be extended in several valuable directions in the future. The photorealistic and immersive novel view rendering reconstructed from our method can be directly leveraged for VR experiences and game development. Compared with traditional content creation pipelines, which require extensive manual modeling and texturing, our Gaussian Splatting results are compatible with popular game engines such as Unity and Unreal Engine. This can be further extended to interactive lighting, textures, and geometry editing in the game engine to support controllable and immersive VR experiences. Besides, our pipeline shows great potential for Sim2Real applications in robotics. Our photorealistic immersive rendering results and the reconstructed meshes could directly benefit the physics-based simulators for robot learning on vision-based navigation and manipulation tasks.

While useful, there is still a problem with these mesh representations – the extracted mesh has no concept of the individual objects in the scene that we as humans might intuitively understand when looking at it. In particular, individual objects being represented in the scene such as doors, chairs, tables, etc. are essentially glued in place in the scene. Object segmentation would allow us to teach our model what it should treat as independent objects. By applying this segmentation to the 3DGS representation before extracting meshes, we could extract meshes per object in the scene, rather than

a singular mesh representing everything in the scene at once. This would allow for a much richer description of the scene geometry and how it should behave.

4.2 Limitations

While our approach gives promising results, there are also many more directions which can be subject to future exploration. We list some of such directions as follows:

- Currently our drones are manually controlled. The drone captures can be automated using SLAM and Reinforcement Learning technology for further lowering the costs and manpower required for carrying out the pipeline.
- Gaussian splatting often suffers from artifacts such as “needles” (Gaussians with low effective rank) and “holes” (low-opacity areas). Some regularization techniques might be able to patch those problems.
- Some works have been able to use generative techniques to improve the quality of 3D reconstruction. We were not able to test diffusion models due to hardware constraints but it is a potential direction for improvement, especially in our case (indoor scene reconstruction) which can contain very specific geometrical and visual priors.

5 Conclusion

We introduce a pipeline for the generation of gaussian splatting-based renderings and meshes of real-world large indoor scenes based on drone captures. We capture numerous novel datasets of indoor scenes, and test different techniques for achieving high 3D reconstruction quality. We also explore innovative methods such as segmentation and post-processing, incorporating them into the pipeline for improved results.

First, we demonstrate that combining consumer-grade drones with 360° cameras provides an efficient data acquisition method that significantly reduces capture time compared to traditional manual approaches. Second, we show that 3D Gaussian Splatting (3DGS) serves as an effective foundation for indoor scene reconstruction, offering real-time rendering capabilities while maintaining high photorealism. Our divide-and-conquer strategy successfully addresses the challenge of processing large-scale environments by breaking them into manageable blocks with strategic overlap. Third, we extend the utility of 3DGS reconstructions through two critical enhancements: (1) mesh extraction techniques such as SuGaR and 2DGS that provide explicit geometry representations, and (2) object segmentation using SAM that identifies individual components within the scene. Our post-processing approach for improving mesh quality, including floor/ceiling detection via histogramming and floater removal, further enhances the practical applicability of these reconstructions.

The resulting pipeline produces not just visually appealing renderings for indoor spaces but also assets that can be directly integrated into simulation environments, AR/VR applications, and game development workflows.

6 Acknowledgments

We would like to acknowledge our advisor Avidesh Zakhor for supporting us with both ideas to explore, and the resources to explore them. We would also like to acknowledge Monica Tang for her work on the segmentation of the scenes we created, we hope that in the future we may be able to integrate this work to create far more robust, segmented meshes. Finally, we would like to acknowledge Ezenbaatar Batjargal for helping us capture our earlier drone flights, and for teaching us how to fly our drone safely.

References

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

- [2] Guédon, A., Lepetit, V. (2024). SuGaR: Surface-Aligned Gaussian Splatting for Efficient 3D Mesh Reconstruction and High-Quality Mesh Rendering. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [3] Huang, J., Yu, H., Chen, X., Geiger, A., Gao, S. (2024). 2D Gaussian Splatting for Geometrically Accurate Radiance Fields. *arXiv preprint arXiv:2401.12345*
- [4] Kerbl, B., Kopanas, G., Leimkühler, T., Drettakis, G. (2023). 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4), 139-1.
- [5] Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, A., Gustafson, L., ... Girshick, R. (2023). Segment Anything. *arXiv preprint arXiv:2304.02643*.
- [6] Kopanas, I., Leimkühler, T., Drettakis, G. (2023). 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*
- [7] Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., Ng, R. (2021). Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1), 99-106.
- [8] Schonberger, J. L., Frahm, J. M. (2016). Structure-from-motion revisited. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 4104-4113).
- [9] Wang, P., Liu, L., Liu, Y., Theobalt, C., Komura, T., Wang, W. (2021). Neus: Learning neural implicit surfaces by volume rendering for multi-view reconstruction. *arXiv preprint arXiv:2106.10689*.
- [10] Li, H., Yi, P., Liu, Y., Zakhori, A. (2023). Scalable MAV Indoor Reconstruction with Neural Implicit Surfaces. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (pp. 1544-1552).
- [11] Chen, Y., Zhang, C., Wang, J., Gao, X., Zakhori, A. (2024). Scalable Indoor Novel-View Synthesis using Drone-Captured 360 Imagery with 3D Gaussian Splatting. In: *Proceedings of IEEE International Conference on Computer Vision (ICCV)* (2024)
- [12] Guédon, A., Lepetit, V. (2024). SuGaR: Surface-Aligned Gaussian Splatting for Efficient 3D Mesh Reconstruction and High-Quality Mesh Rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 5354-5363).
- [13] Huang, B., Yu, Z., Chen, A., Geiger, A., Gao, S. (2024) 2D Gaussian Splatting for Geometrically Accurate Radiance Fields. In *ACM SIGGRAPH 2024 Conference Papers*.
- [14] M. Ye, M. Danelljan, F. Yu, and L. Ke, "Gaussian Grouping: Segment and Edit Anything in 3D Scenes," *arXiv.org*, 2023. <https://arxiv.org/abs/2312.00732>
- [15] Chang, A., Dai, A., Funkhouser, T., Halber, M., Niessner, M., Savva, M., Zhang, Y. (2017). Matterport3d: Learning from rgb-d data in indoor environments. *arXiv preprint arXiv:1709.06158*.
- [16] Fisher, A., Cannizzaro, R., Cochrane, M., Nagahawatte, C., Palmer, J. L. (2021). ColMap: A memory-efficient occupancy grid mapping framework. *Robotics and Autonomous Systems*, 142, 103755.
- [17] Leroy, V., Cabon, Y., Revaud, J. (2024, September). Grounding image matching in 3d with mast3r. In *European Conference on Computer Vision* (pp. 71-91). Cham: Springer Nature Switzerland.
- [18] Z. Li, R. Tucker, F. Cole, Q. Wang, L. Jin, V. Ye, A. Kanazawa, A. Holynski, and N. Snavely, "MegaSaM: Accurate, Fast, and Robust Structure and Motion from Casual Dynamic Videos," *arXiv preprint arXiv:2401.12588*, 2024.
- [19] A. Brunton, S. Wuhler, C. Shu, P. Bose and E. D. Demaine, "Filling holes in triangular meshes by curve unfolding," 2009 IEEE International Conference on Shape Modeling and Applications, Beijing, China, 2009, pp. 66-72, doi: 10.1109/SMI.2009.5170165. keywords: Filling;Geometry;Solid modeling;Shape;USA Councils;Geometrical optics;Laser modes;Computational Geometry;Object Modeling;Hole Filling;Curve Unfolding,